

# Object Oriented Concepts || P4

---

Rahul Guha || SME || Java FS Training || HCL

# Errors in Java

## Introduction

- An Error in Java is a serious problem that occurs at runtime (during the execution of a program).
- Errors usually happen due to system-level issues (like memory shortage or JVM crash) rather than problems in the application logic.
- Errors are beyond the control of the programmer and generally cannot be recovered from within the code.

## For Example : Stack Overflow Error

Occurs when there is deep or infinite recursion.

Program:

```
public class StackErrorExample
{
    public static void main(String[] args)
    {
        main(null); // infinite recursive call
    }
}
```

# Errors in Java

## Out Of Memory Error

Occurs when the JVM runs out of memory to allocate new objects.

### Program:

```
import java.util.ArrayList;

public class MemoryErrorExample
{
    public static void main(String[] args)
    {
        ArrayList list = new ArrayList<>();
        while (true)
        {
            list.add(new int[1000000]); // keeps consuming memory
        }
    }
}
```

## Virtual Machine Error

Thrown when the Java Virtual Machine (JVM) encounters a serious internal problem.

### Example:

- JVM crash due to insufficient resources.
- Rare, but indicates JVM is in unstable state.

# Java Exception Handling

**Exception handling** in Java is a mechanism used to handle exceptional conditions that occur during program execution. It helps prevent the program from terminating unexpectedly and allows developers to provide appropriate recovery or error-handling logic.

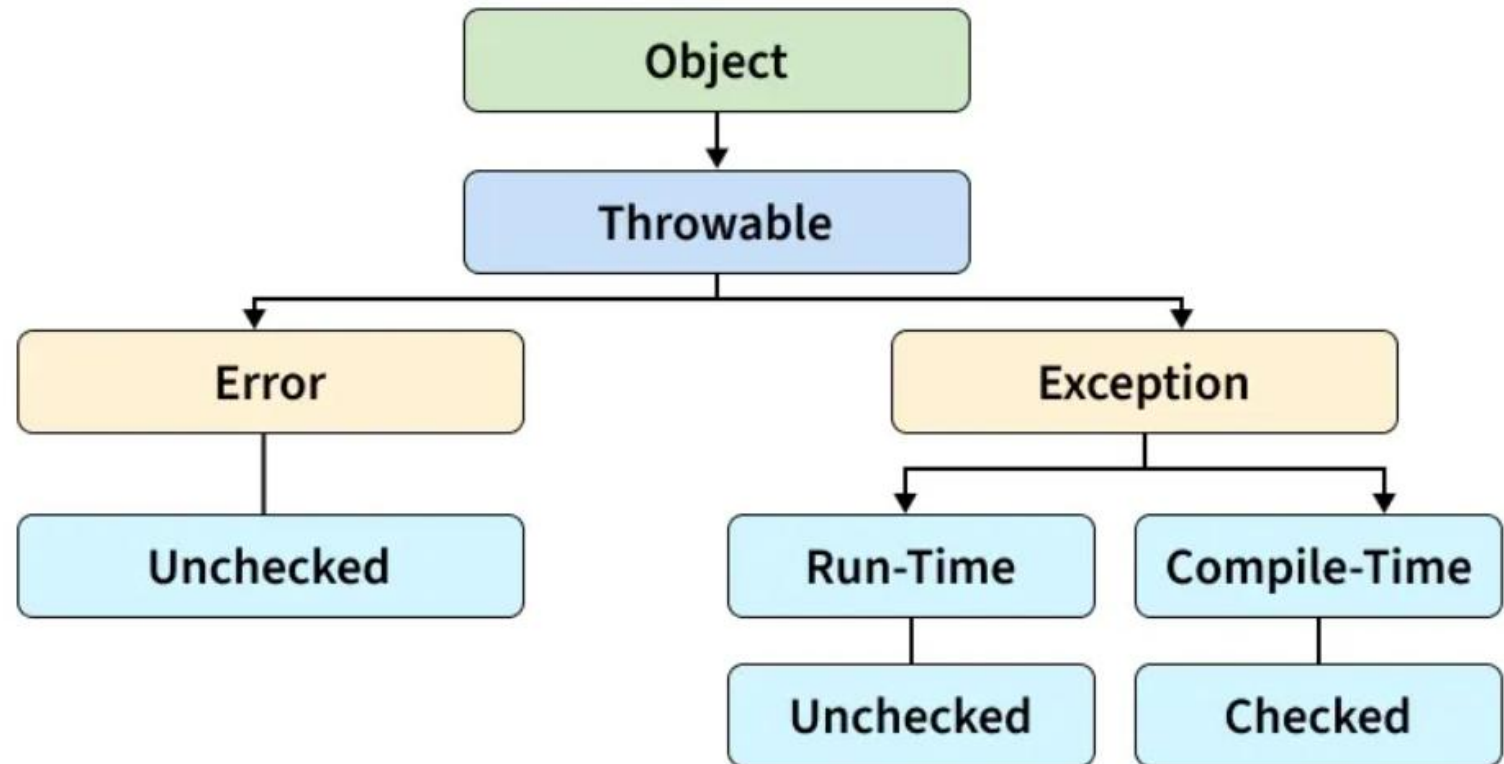
- Separates error-handling code from normal program logic.
- Helps maintain the normal flow of program execution.
- Supports handling of both checked and unchecked exceptions.

| Types of Errors in Java |                                                            |
|-------------------------|------------------------------------------------------------|
| Error Type              | Description                                                |
| Compile-Time Error      | Detected by the compiler. Prevents code from running.      |
| Runtime Error           | Occurs while the program is running. Often causes crashes. |
| Logical Error           | Code runs but gives incorrect results. Hardest to find.    |

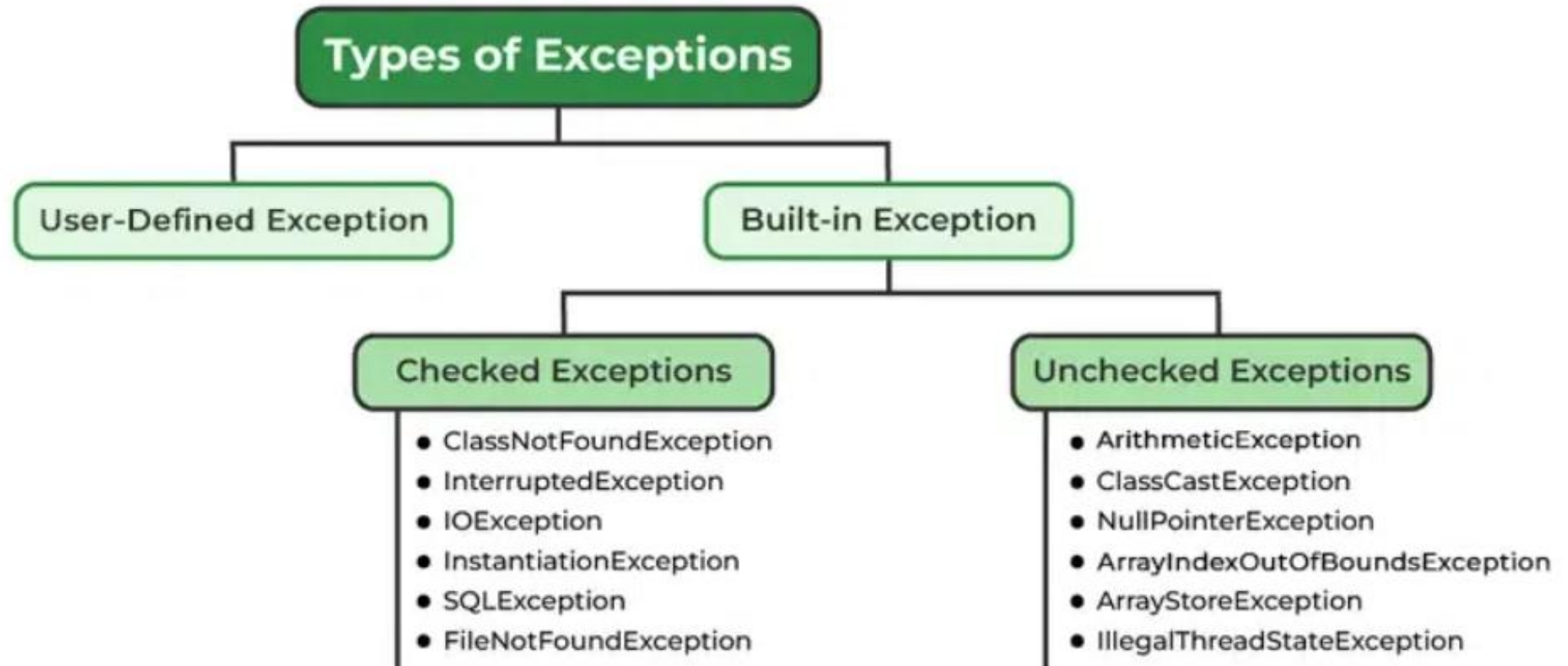
# Java Exception Hierarchy

In Java, all exceptions and errors are subclasses of the Throwable class. It has two main branches

- Exception.
- Error



# Java Exception Hierarchy



# Java Exception Handling

## 2. Why Exception Handling is Required?

Exception handling helps us:

- Prevent abnormal termination of programs
- Handle runtime errors
- Display meaningful error messages
- Maintain normal program flow
- Separate error-handling code from normal code

| Exception                      | Example                             |
|--------------------------------|-------------------------------------|
| ArithmeticException            | Division by zero                    |
| ArrayIndexOutOfBoundsException | Invalid array index                 |
| NullPointerException           | Using a null reference              |
| NumberFormatException          | Invalid string-to-number conversion |
| FileNotFoundException          | File does not exist                 |
| IOException                    | Input/output error                  |

# Java Exception Handling

**try Block:** The try block contains the code that may generate an exception.

```
try {  
    // risky code  
}
```

However, a try block cannot normally be used alone. It must be followed by catch or finally.

**catch Block:** The catch block is used to handle an exception generated inside the try block.

```
try {  
    // risky code  
}  
catch (ExceptionType e) {  
    // handling code  
}
```

# Java Exception Handling

**What is finally?** The finally block contains code that generally needs to execute after the try/catch processing, whether or not an exception occurs. It is commonly used for cleanup operations, such as closing files or releasing resources.

```
try {  
    // code  
}  
  
catch (Exception e) {  
    // handling  
}  
  
finally {  
    // cleanup code  
}
```

# Java Exception Handling

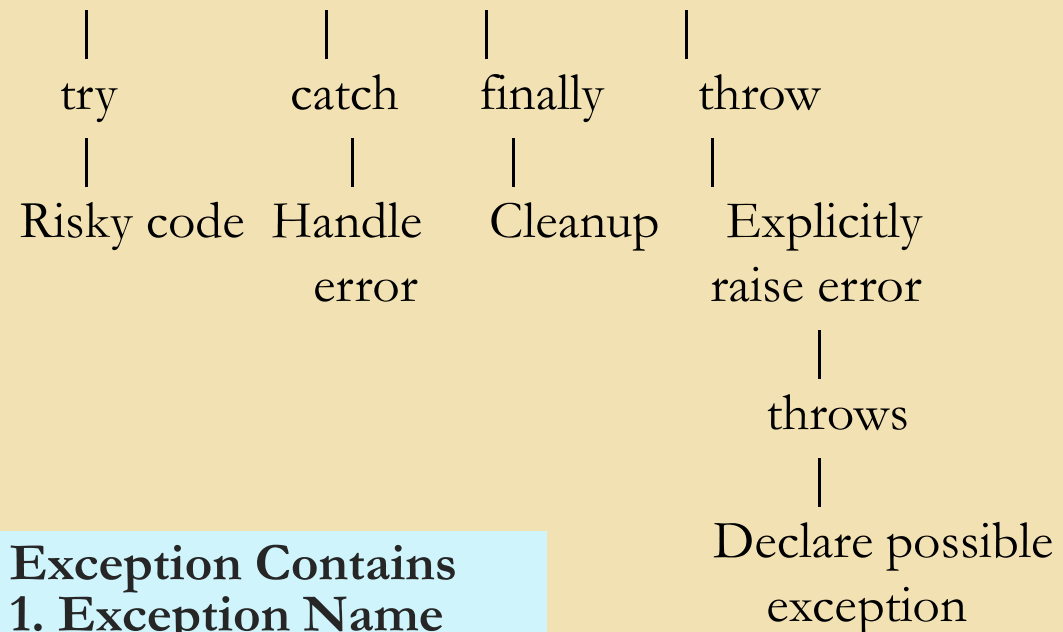
**throw Keyword** The throw keyword is used to explicitly throw an exception.  
`throw new ExceptionType("message");`

throw is used to create/raise an exception manually.

**throws Keyword :** The throws keyword is used in a method declaration to indicate that the method may throw one or more exceptions.

```
returnType methodName() throws  
ExceptionType {  
    // code  
}
```

## Exception Handling



**Exception Contains**

1. Exception Name
2. Description
3. Stack Trace(JVM)

# Java Exception Handling

| <b>throw</b>                                 | <b>throws</b>                                      |
|----------------------------------------------|----------------------------------------------------|
| Used to explicitly throw an exception        | Used to declare possible exceptions                |
| Used inside a method/block                   | Used in method declaration                         |
| Throws one exception at a time               | Can declare multiple exceptions                    |
| Followed by an exception object              | Followed by exception class names                  |
| Example: <code>throw new Exception();</code> | Example: <code>void test() throws Exception</code> |

# Java Exception Handling

| Feature            | final                                               | finally                                          | finalize()                                              |
|--------------------|-----------------------------------------------------|--------------------------------------------------|---------------------------------------------------------|
| Type               | Keyword                                             | Block                                            | Method                                                  |
| Main purpose       | Restricts modification                              | Executes cleanup code after try/catch processing | Historically used for cleanup before garbage collection |
| Used with          | Variable, method, class                             | try-catch                                        | Object                                                  |
| Related to         | OOP restrictions                                    | Exception handling                               | Garbage Collection                                      |
| Can be overridden? | Depends on usage; final method cannot be overridden | Not applicable                                   | Historically could be overridden                        |
| Can be inherited?  | Final class cannot be extended                      | Not applicable                                   | Method was inherited like other methods                 |
| Syntax             | final int x = 10;                                   | finally { }                                      | protected void finalize()                               |
| Execution          | Depends on variable/method/class                    | Normally executes after try/catch                | <b>Not guaranteed</b>                                   |
| Modern Java status | Fully supported                                     | Fully supported                                  | <b>Deprecated and should not be used</b>                |